```
| SetTickets
| UnholdTicket
```

When these two pieces are combined, TypeScript treats `if` or `switch` statements that check for the value of the discriminant as a type guard. In our actual reducer function, we have `switch (action.type)` and then a lot of clauses like `case "setTicketsToBuy":`. TypeScript treats each of those like a type guard, meaning that inside each clause, TypeScript uses the value of the literal to infer which member type of the union is being used. As we saw when we originally wrote this code, we can refer to attributes of that interface that are specific to that one particular type. (Your IDE might helpfully annotate the code to tell you where TypeScript has used a type guard to narrow the type of the action to a specific interface.)

Exhaustiveness Checking



One thing you might like to have that is not very convenient given the way we have TypeScript set up is *exhaustiveness checking*. It'd be nice to know that if we have an `if` or `switch` statement based on a literal type or a discriminated union, our code can handle any of the possible values. Unfortunately, the way TypeScript handles `null` and `undefined` makes this challenging. By default, any TypeScript value can be `null` or `undefined`, and as a result, it's hard to get the compiler to prove that a `switch` statement is exhaustive.